Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015118**	**Course Name:**	**Artificial Intelligence & Soft Computing Lab**

Name of Student:	**Shreyas Sawant**
Roll No. :	**78**
Experiment No.:	**2**
Title of the Experiment:	**Implement Logic Programming Using PROLOG**
Date of Performance:	**27/7/2026**
Date of Submission:	**3/8/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :

Signature :

Date :

Aim: To write and execute basic programs in PROLOG.

Objectives:

After completing this experiment, the student will be able to:

1. To understand the fundamentals of logic programming.
2. To learn the basic syntax and structure of PROLOG programs.
3. To write facts, rules, and queries in PROLOG.
4. To execute and verify simple PROLOG programs.
5. To understand the concept of logical reasoning using PROLOG.

Tools Required:

- **Hardware:**
  - Desktop/Laptop
- **Software:**
  - SWI-PROLOG / GNU PROLOG
  - Windows/Linux Operating System

Theory:

PROLOG (Programming in Logic) is a declarative programming language widely used in the field of Artificial Intelligence for knowledge representation, logical reasoning and problem-solving. Unlike procedural programming languages, PROLOG focuses on describing facts and relationships using logical statements rather than specifying step-by-step instructions.

A PROLOG program consists of facts, rules, and queries. Facts represent known information, rules define logical relationships between facts, and queries are used to retrieve information or verify logical conditions. PROLOG uses an inference engine based on pattern matching and backtracking to derive solutions automatically. Due to its powerful reasoning capabilities, PROLOG is commonly used in expert systems, natural language processing, theorem proving and intelligent decision-support systems

Procedure:

1. Install and launch the SWI-PROLOG environment.
2. Create a new PROLOG source file.
3. Write the required facts and rules using proper PROLOG syntax.
4. Save and load the program into the PROLOG interpreter.
5. Execute various queries to verify the program.
6. Observe the generated outputs and validate the results.
7. Record the observations and conclusions.

Description of the Experiment:

This experiment introduces the fundamentals of logic programming using PROLOG. Basic programs are developed by defining facts and rules representing real-world relationships, followed by executing queries to verify the logical conclusions generated by the PROLOG inference engine.

The experiment provides practical exposure to knowledge representation and logical reasoning, enabling an understanding of how AI systems perform inference and solve problems using declarative programming techniques.

Detailed Description:

PROLOG programs are developed by representing knowledge in the form of facts and logical rules. Facts describe basic information about objects or relationships, while rules define logical conditions that derive new information from existing facts. Queries are then executed to verify whether a particular condition is true or false based on the available knowledge base.

In this experiment, simple PROLOG programs are written and executed to understand the concepts of facts, rules and logical inference. The outputs obtained from different queries demonstrate the reasoning capability of PROLOG and illustrate its application in solving Artificial Intelligence problems through logical knowledge representation

OUTPUT:.

```
5 ?- consult(['C:/Users/student/Desktop/Ai_powered_ICU_Patient_Monitor.p1.pl']).
6 ?-.patient(X).
X = john .

7 ?- critical(X).
X = john .

8 ?- icu_alert(X).
X = john .

9 ?- listing(patient).
patient(john).
patient(emma).
patient(david).

true.
```

Conclusion:

Basic logic programs were successfully written and executed using PROLOG. The experiment demonstrated the concepts of knowledge representation, logical reasoning, and query execution through facts, rules, and inference mechanisms.

Questions to be Answered by Students:

1. Which PROLOG statements were defined as facts in the program?

The following statements were defined as **facts** to represent patient information and vital signs:

```
patient(j
ohn).

patient(em
ma).

patient(davi
d).
heart_rate(john,
high).

heart_rate(emma,
normal).

heart rate(david,
low).

oxygen(john,
low).

oxygen(emma,
normal).

oxygen(david,
low).

blood pressure (john,
high).

blood pressure(emma,
normal).
```

```
blood pressure(david,
low).

temperature(john,
fever).

temperature(emma,
normal).

temperature(david,
normal).
```

These facts represent the knowledge base of the ICU patient monitoring system.

2. Which rules were implemented to establish logical relationships?

The following rules were implemented:

```
critical(Patient)
:-
    oxygen(Patient,
    low).

critical(Patient)
:-
    heart_rate(Patient,
    high).

critical(Patient
) :-
    heart rate (Patient,
    low).

icu alert(Patient) :-
    critical
    (Patient),
    blood_pressure(Patient,
    high).

icu alert(Patient) :-
    critical(Patie
    nt),
    blood pressure(Patient,
```

```
    low).

stable(Patie
nt) :-
    heart_rate(Patient,
    normal),
    oxygen(Patient,
    normal),
    blood pressure(Patient, normal),
    temperature(Patient, normal).

monitor(Patient)
:-
    critical
    (Patient).
```

These rules infer whether a patient is **critical**, **stable**, requires an **ICU alert**, or needs **continuous monitoring**.

3. What output was obtained for the executed queries?

QUERY	OUTPUT
patient(X).	X = john; X = emma; X = david.
critical(X).	X = john; X = david.
stable(X).	
icu_alert(X).	
monitor(john).	

X
= emma.

X = john; X =
david.

t
r
u
e
.

4. State two observations obtained from the execution of the PROLOG program.

The PROLOG inference engine successfully identified critical patients based on abnormal vital signs without explicitly storing the "critical" status as a fact.

Logical rules enabled automatic generation of ICU alerts and monitoring decisions,

demonstrating PROLOG's capability for knowledge representation and reasoning in AI-based healthcare applications.